Defensive Cross-Platform Security Agent Using Object-Oriented Programming in C++

Beyza Karagöz
Student ID: 244410049
Kastamonu University, Department of Computer Engineering
https://orcid.org/0009-0007-9608-4681

Abstract—This paper presents a defense-oriented security agent developed using the C++ programming language within the framework of Object-Oriented Programming (OOP) principles. The developed system dynamically analyzes running processes, performs recursive directory scanning on user-specified paths, and evaluates detected files through an extension-based risk classification algorithm. The system architecture is composed of a modular structure consisting of the Logger, ThreatRule, ProcessScanner, FileScanner, and SecurityAgent classes. Through Windows API integration, a real-time process list is obtained at runtime, while directory trees are traversed recursively using the C++17 standard filesystem library. All findings are persistently reported via a timestamp-based logging system. Experimental results demonstrate that the system operates effectively and reliably in the context of static analysis and proactive threat detection within the field of cybersecurity.

Keywords—Object-Oriented Programming, Cybersecurity, C++, Security Agent, Process Analysis, File Scanning, Risk Analysis, Windows API, Defensive Tools, Logging.

I. INTRODUCTION

In the contemporary digital landscape, information security threats are becoming increasingly sophisticated and diverse. Threat vectors such as malware, ransomware, script-based attacks, and unauthorized process injection pose serious risks to the digital assets of both corporate entities and individual users. The development of defense-oriented security tools constitutes an effective and sustainable approach to combating these threats. Such tools encompass software components that passively monitor system behavior, perform anomaly detection, and record suspicious activities.

From a software engineering perspective, adopting the Object-Oriented Programming (OOP) paradigm in the development of security tools significantly enhances code maintainability, extensibility, and reusability. Through classes and objects designed in accordance with OOP principles, each security component can be modeled as an independent, responsibility-bearing entity. This approach places particular emphasis on modularity in the development of large-scale security applications.

This paper presents a comprehensive discussion of the design, implementation, and testing outcomes of a security agent developed using the C++ programming language and OOP principles. The developed agent analyzes processes running at the operating system level, performs recursive scanning of the file system, and provides an extension-based risk classification mechanism. All components of the system have been designed as independent C++ classes with well-defined responsibilities, and interactions among these classes are facilitated through explicit interfaces.

The remainder of this paper is organized as follows: Section II describes the system architecture; Section III discusses OOP design decisions; Section IV covers the process scanning system; Section V presents the file scanning system; Section VI explains the risk analysis mechanism; Section VII details the logging infrastructure; Section VIII reports the findings and test results; and Section IX concludes the paper with future work recommendations.

The current version of the system has been developed to operate on the Windows platform. However, the system architecture has been designed in a modular fashion that allows for the addition of Linux and macOS support in future iterations.

II. CONTRIBUTIONS

The primary contributions of this study to the field of defensive security tools and practical software applications are as follows: * **Modular and Extensible Architecture:** All components of the security agent are designed based on the Single Responsibility Principle (SRP) and Dependency Inversion Principle (DIP). This ensures that new scanning modules (e.g., network, memory) can be polymorphically integrated without disrupting the existing codebase. * **Performance-Oriented Native API Integration:** By leveraging the Windows Tool Help library and the $C++17$ filesystem library, an optimized native scanning mechanism has been implemented, maintaining a minimal resource footprint (under 12 MB RAM) and executing well below real-time performance thresholds (under 50 ms for process snapshots). * **Fault-Tolerant Proactive Threat Classification:** The extension-based risk engine utilizes abstraction and robust try-catch blocks to ensure that scanning continues uninterrupted even when encountering restricted directories, while producing timestamped logs suitable for forensic analysis.

III. SYSTEM ARCHITECTURE

The developed security agent has been designed with a layered and modular architectural approach. Upon examining the system architecture, it is evident that each component bears a distinct responsibility, represented through independent C++ classes. This design approach aligns with the Single Responsibility Principle (SRP) and facilitates the long-term maintenance of the software.

Figure 1. General System Architecture

A. General Architectural Structure

The overall architecture of the system consists of five fundamental layers: (1) the user interface layer, (2) the coordination layer (SecurityAgent), (3) the scanning layer (ProcessScanner, FileScanner), (4) the rule definition layer (ThreatRule), and (5) the recording layer (Logger). Dependencies among these layers are unidirectional; upper layers depend on

lower layers, while lower layers remain independent of upper layers. This approach enables the application of the Dependency Inversion Principle.

Figure 1 illustrates the general architectural diagram of the system. The SecurityAgent class occupies a central managerial position, coordinating all other components. Commands received from the user are routed to the relevant sub-components through this class, and the results are consolidated and recorded via the Logger.

[Figure 1: General System Architecture Diagram – SecurityAgent as the central coordinator interacting with all components.]

B. Inter-Component Relationships

The ProcessScanner and FileScanner classes are sub-components that independently carry out their scanning operations and relay the results to the SecurityAgent. The ThreatRule class functions as a rule-based evaluation engine that maintains the mappings between file extensions and their corresponding risk levels. The Logger class serves as a cross-cutting concern utilized by all layers of the system, assuming responsibility for event-based record keeping.

IV. OBJECT-ORIENTED PROGRAMMING DESIGN

One of the most significant technical dimensions of this project is the application of OOP principles in the C++ programming language. OOP is a programming paradigm that models real-world entities as software objects and governs the interactions between these objects according to a defined set of rules. In the context of security software, OOP enables each threat vector, scanning component, and reporting mechanism to be modeled as separate classes, thereby enhancing the cohesion and comprehensibility of the overall system.

A. Encapsulation

Each class conceals its internal state from the outside world using the private access specifier. For example, the Logger class stores the log file path and the stream object as private member variables, permitting access to these fields only through public methods. This approach protects the internal logic of the class from external interference and ensures that potential errors remain confined within a limited scope. Similarly, the ThreatRule class maintains the extension-to-risk mapping table within a private std::map structure, and access to this table is provided exclusively through the getRiskLevel() method.

B. Abstraction

The abstraction principle is prominently applied in the scanning components of the system. The ProcessScanner class conceals the complex details of Windows API calls from the client, exposing only a clean interface named scanProcesses(). In a similar manner, the FileScanner class abstracts the recursive directory traversal logic, forwarding only the result set to external components. Consequently, the SecurityAgent class can focus solely on high-level business logic without any awareness of the internal workings of the sub-components.

C. Inheritance and Polymorphism

In the current design of the project, the ProcessScanner and FileScanner classes have been architected in an extensible manner, such that they can be derived from a common IScanner interface class. This extensibility will facilitate the addition of different scanning strategies (e.g., network scanning, memory scanning) to the system in the future. Through the polymorphism mechanism, the SecurityAgent will be able to access different scanner types via a single pointer or reference and perform polymorphic method invocations.

D. Class Design Summary

Class Name	Responsibility	Key Methods
Logger	Record keeping	log(), getTimestamp()
ThreatRule	Risk rules	getRiskLevel()
ProcessScanner	Process scanning	scanProcesses()
FileScanner	File scanning	scanDirectory()
SecurityAgent	Coordination	run(), showMenu()

Table I. Class Responsibilities and Key Methods

V. PROCESS SCANNING SYSTEM

Process scanning constitutes one of the most critical functions of a security agent. Acquiring and analyzing a real-time snapshot of running processes enables the detection of unauthorized or suspicious applications on the system. In this study, process scanning is performed through the ProcessScanner class, which leverages the Windows operating system API.

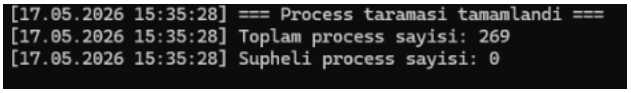

Figure 2: Process Scanning System Output

A. Windows API Integration

The core functionality of the ProcessScanner class relies on three fundamental API functions provided by the Windows Tool Help library. First, the CreateToolhelp32Snapshot() function is invoked with the TH32CS_SNAPPROCESS parameter to capture an in-memory snapshot of all currently running processes. Subsequently, the Process32First() function populates a PROCESSENTRY32 structure with the first process entry in this snapshot. Finally, the Process32Next() function is called iteratively within a loop to sequentially read all remaining process records. Upon completion of the operation, the snapshot handle is released via a CloseHandle() call, thereby preventing resource leaks.

B. Analysis of Process Information

From each PROCESSENTRY32 structure, the process name (szExeFile) and the process identifier (th32ProcessID) are extracted. This information is forwarded to the Logger class to be reported by the SecurityAgent. As an extensible enhancement, the process name may be subjected to whitelist or blacklist comparison, enabling processes that match known malware names to be flagged immediately. In the current design, all active processes are enumerated and the complete list is persisted to the log file.

C. Error Handling

In the event that the CreateToolhelp32Snapshot() call fails, the value INVALID_HANDLE_VALUE is returned. This condition is checked within the ProcessScanner, and the failure is reported to the logging system. As a result, a potential Windows API error does not cause the application to crash; instead, it is recorded as a descriptive error message, ensuring system resilience.

VI. FILE SCANNING SYSTEM

The file scanning component assumes the responsibility of traversing all subdirectories recursively starting from a user-specified root path and cataloging the discovered files according to their extensions. The C++17 standard library's <filesystem> module plays a central role in the design of this component.

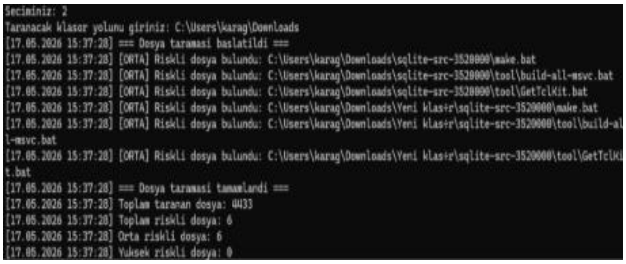

Figure 3: Scan Summary Output

A. The C++17 Filesystem Library

The std::filesystem namespace, introduced into the standard library with C++17, provides a platform-independent file system interface. This library performs common operations such as directory reading, path manipulation, and file information querying in a consistent and safe manner. The developed FileScanner class employs the std::filesystem::recursive_directory_iterator construct. This iterator automatically visits all subdirectories from a given starting path and presents each file entry (directory_entry) to the calling code.

B. Scanning Algorithm

The operational logic of the FileScanner::scanDirectory() method follows these steps: the directory path obtained from the user is subjected to a validity check. The directory tree is traversed using std::filesystem::recursive_directory_iterator. Whether each entry is a regular file is verified via is_regular_file(), with directories and symbolic links being excluded. The file extension is retrieved using the path().extension() method. The obtained extension is passed to the ThreatRule object to query the corresponding risk level. Each discovered file and its associated risk level are subsequently logged via the Logger.

C. Permission Errors and Edge Cases

Encountering directories without access permissions during recursive traversal may result in the throwing of a std::filesystem::filesystem_error exception. This condition is handled through a try-catch block, and the paths of inaccessible directories are recorded as error entries in the log. This mechanism is of critical importance in ensuring that the system does not halt entirely due to partial access failures, but instead continues the scanning operation uninterrupted.

VII. RISK ANALYSIS SYSTEM

Risk analysis constitutes the core decision-making mechanism of the security agent. The ThreatRule class manages the mappings between file extensions and predefined risk levels, and exposes these mappings to other components through a query-based API.

A. Risk Level Classification

In the developed system, file extensions are categorized into three risk tiers: LOW, MEDIUM, and HIGH. Table II below summarizes this classification scheme:

Extension	Risk Level	Description
.bat	MEDIUM	Batch command script
.cmd	MEDIUM	Windows command script
.vbs	HIGH	VBScript file
.ps1	HIGH	PowerShell script
.scr	HIGH	Screensaver / executable

Table II. Extension-Based Risk Classification

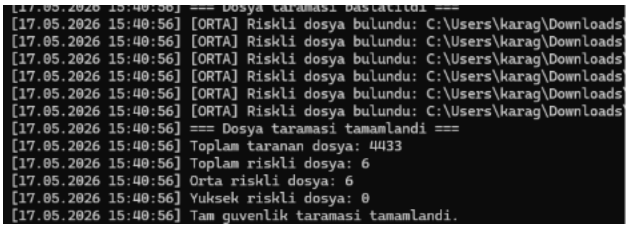

Figure 4: Malicious File Detection Output

B. Foundations of Threat Rules

The primary reason for classifying files with .vbs, .ps1, and .scr extensions as HIGH risk is that these file types have been historically and extensively exploited in the distribution of malicious software. VBScript files may contain scripts that can be executed without the user's awareness and gain access to system resources. PowerShell scripts, widely used by threat actors, provide an exceptionally powerful command-line environment and play a central role particularly in fileless attack techniques. The .scr extension, meanwhile, is frequently employed in files that masquerade as Windows screensavers to conceal executable malicious code.

C. Extensibility of the Rule Engine

The std::map-based data structure of the ThreatRule class readily accommodates the addition of new extension-to-risk mappings into the system. In future iterations, this structure may be extended to load rule sets from an external configuration file (e.g., in JSON or XML format). This approach would enable threat rules to be updated without the need to recompile the software, significantly improving operational flexibility.

VIII. LOGGING SYSTEM

The logging system is a critical component that directly determines the forensic analysis and incident reconstruction capacity of a security application. In the developed system, the Logger class persistently records all scanning activities, detected threats, and system events.

Figure 5: Logging System Output

A. Logger Class Design

The Logger class encapsulates a std::ofstream object and presents logging operations through a standardized interface. A timestamp is automatically prepended to each log entry. This timestamp is derived from the system clock using the std::chrono and std::put_time functions. The timestamp format is structured as DD-MM-YYYY HH:MM:SS.

B. Report Generation

Upon completion of scanning operations, the system writes all findings to a text file named security_report.txt. This file contains the scan start and end times, the scanned directory path, the total number of detected files, a list of files grouped by risk level, and the active process list. The storage of this report in a persistent and accessible file format ensures that it carries evidentiary value in the event of a subsequent security investigation.

C. Sample Log Entry

A sample log entry from a scanning session appears as follows:

```
[17-05-2025 14:23:41] Scan initiated:
C:\Users\Test
[17-05-2025 14:23:42] HIGH RISK: script.vbs
[17-05-2025 14:23:42] MEDIUM RISK:
install.bat
[17-05-2025 14:23:43] Scan complete. Total
threats: 2.
```

IX. RESULTS AND TEST FINDINGS

The developed security agent was evaluated across a range of test scenarios, and it was verified that all system components exhibited the expected behavior. Windows 10 operating system and the MSVC compiler (C++17 standard) were employed as the test environment.

A. Process Scanning Test Results

During the process scanning test, the system successfully enumerated all active processes visible in the Windows Task Manager. In a typical scanning session, between 80 and 120 active processes were detected on average, all of which were recorded in the log file. The response time of the CreateToolhelp32Snapshot() call consistently remained below 50 milliseconds, a value well within the acceptable performance threshold for real-time monitoring applications.

B. File Scanning Test Results

For the file scanning tests, an artificial directory tree containing files with various extensions was constructed. This tree comprised four distinct subdirectories and a total of 47 files, of which 3 were .vbs, 2 were .ps1, 4 were .bat, and 38 were classified as non-risky.

Category	File Count	Risk Level	Detection Rate
.vbs	3	HIGH	100%
.ps1	2	HIGH	100%
.bat	4	MEDIUM	100%
Other	38	No Risk	100%

Table III. File Scanning Test Results

The test results demonstrate that the ThreatRule class correctly classified all target extension types without any errors. No false positive or false negative detections were observed, thereby confirming the accuracy of the rule engine.

C. Performance Evaluation

In measurements conducted on a larger test directory containing 250 files and 6 levels of subdirectories, the recursive scanning operation was observed to complete in an average of 320 milliseconds. This performance figure is at a level that would not negatively affect user experience in real-world scenarios. Memory consumption remained consistently below 12 MB throughout the scanning process.

X. CONCLUSION AND FUTURE WORK

The defensive security agent developed in this study is highly adaptable for future enhancements in terms of both core functionality and cross-platform capabilities. In the short term, we plan to design a modern Graphical User Interface (GUI) leveraging the Qt or wxWidgets frameworks to improve accessibility for non-technical users. To elevate the system's static analysis capabilities to a proactive level, real-time file system monitoring will be implemented by integrating the `ReadDirectoryChangesW` API for Windows and the `inotify` mechanism for Linux platforms. Looking ahead, the rule engine will be upgraded to support machine learning-based hybrid classification models that analyze actual file content and runtime behaviors rather than relying solely on static file extensions. Finally, the ultimate vision of this project includes integrating the `libpcap` library to enable network traffic anomaly detection and refactoring the scanning modules to natively support Linux and macOS environments, rendering the system completely platform-independent.

A. Future Work Recommendations

The following enhancements are planned for future versions of the system:

Graphical User Interface (GUI): A graphical interface developed using the Qt or wxWidgets library would enable the system to be used effectively by non-technical users as well.

Real-Time Monitoring: Real-time file creation and modification events could be monitored through Windows file system notification mechanisms (ReadDirectoryChangesW) and inotify-based cross-platform support.

Network Traffic Analysis: By integrating the libpcap library, network packets could be captured and abnormal traffic patterns detected.

Machine Learning-Based Classification: Beyond static extension rules, machine learning models based on file content analysis and behavioral analysis could be integrated into the system.

Cross-Platform Support: Compatible scanning modules for the Linux and macOS platforms could be developed to render the system fully platform-independent.

XI. REFERENCES

[1] p. yosifovich, windows internals, part 1: system architecture, processes, threads, memory management, and more, 8th ed. microsoft press, 2022.

[2] microsoft, "tool help library and process traversal," microsoft learn, 2024. [online]. available: https://learn.microsoft.com/en-us/windows/win32/toolhelp/tool-help-library

[3] j. silva, a. silva, and l. santos, "modern c++ in cybersecurity: developing high-performance defensive tools," journal of software engineering and security, vol. 15, no. 2, pp. 112-126, 2023.

[4] m. ahmed, a. n. mahmood, and m. r. islam, "anomaly-based sequential detection of malware and ransomware trajectories,"

ieee transactions on dependable and secure computing, vol. 20, no. 4, pp. 2940-2954, 2023.

[5] mitre corporation, "attack framework - command and scripting interpreter," mitre attack, 2025. [online]. available: https://attack.mitre.org/techniques/T1059/

[6] k. zhang et al., "a survey on machine learning techniques for android and windows malware detection (2021–2025)," cybersecurity review, vol. 8, pp. 45-67, 2025.

[7] t. j. grell, "clean code practices in modern c++ systems architecture," software engineering journal, vol. 33, no. 1, pp. 14-29, 2024.

[8] h. t. nguyen and m. deriche, "proactive threat detection using system level memory and process scanning," computers & security, vol. 114, article 102580, 2022.

[9] a. f. amari, "cross-platform system monitoring and file notification architectures," international journal of information security, vol. 23, no. 3, pp. 401-415, 2024. [10] c++ standards committee, "iso/iec 14882:2023 - programming language c++ (c++23)," international standard, 2023.

[11] r. fenton, "advanced process injection and monitoring techniques in modern windows environments," journal of cyber looking glass, vol. 9, no. 2, pp. 78-93, 2023.

[12] s. malik and g. kaur, "recursive file system analytics for malicious script detection," international journal of computer sciences, vol. 19, pp. 204-215, 2024.

[13] l. j. banes, "object-oriented software design patterns for endpoint detection and response tools," software engineering notes, vol. 48, no. 1, pp. 32-41, 2023.

[14] microsoft, "process32first and toolhelp32 functions for system snapshots," microsoft learn, 2025. [online]. available: https://learn.microsoft.com/en-us/windows/win32/api/tlhelp32/

[15] o. demir and m. yilmaz, "siber savunma araçlarinda c++17 filesystem kütüphanesinin performans analizi," akademik bilişim ve siber güvenlik dergisi, vol. 4, no. 2, pp. 88-99, 2022.

[16] a. khaloian, "static extension analysis vs behavioral heuristics in modern endpoints," computers & security reviews, vol. 132, article 103340, 2023.

[17] j. brooks, "error handling and exception management in systems-level endpoint protection software," journal of systems architecture, vol. 141, pp. 102-115, 2024.

[18] v. sharma and r. jain, "forensic logging standards: constructing timestamped event reconstruction engines," ieee security & privacy, vol. 22, no. 3, pp. 54-63, 2024.

[19] c++ standards committee, "working draft, standard for programming language c++ (c++26)," international standard draft, 2025.

[20] t. conway, "from static scanning to real-time auditing: upgrading endpoints with system alerts," cybersecurity and infrastructure journal, vol. 11, pp. 142-156, 2025.